Smart Manufacturing & Predictive Maintenance

Power BI Dashboard on a SQL Server Star Schema

Graduation Project Documentation - Digital Egypt Pioneers Initiative (DEPI)

Data source: Kaggle - Smart Manufacturing & Predictive Maintenance Dataset (100,000 rows x 72 columns, 5,000 machines)

Author: Eman Sayed Seddik

Section	Content
1	Project overview and objectives
2	Project planning
3	Stakeholder analysis
4	Database design
5	UI/UX design
6	Data findings and limitations
7	How to reproduce

Reading note: the stakeholders in section 3 are assumed roles for a realistic factory scenario - the dataset is synthetic, so there is no real client. The wireframes in section 5 are design mock-ups and should be replaced or complemented by screenshots of the finished Power BI report.

1. Project overview and objectives

Factories collect thousands of sensor readings but often use them only after a machine has stopped. This project builds a monitoring dashboard on top of a clean SQL star schema so that maintenance and plant teams can see, in one place, how often machines fail, which failure types occur, how healthy the fleet is, and what maintenance and downtime cost.

Problem statement

Plant and maintenance managers need one trusted view of machine health, failures, downtime and maintenance effort across 10 factories, instead of reading a 72-column flat file.

Objectives

- Design and load a dimensional (star) data model from the flat Kaggle file with no loss of rows or totals.
- Deliver a 4-page Power BI report: Executive Overview, Machine Health & Risk, Maintenance & Cost, Sensors & Anomalies.
- Define a transparent risk tier, because the source column `maintenance_priority` is constant.
- Document honestly what the data can and cannot support (section 6).

Scope

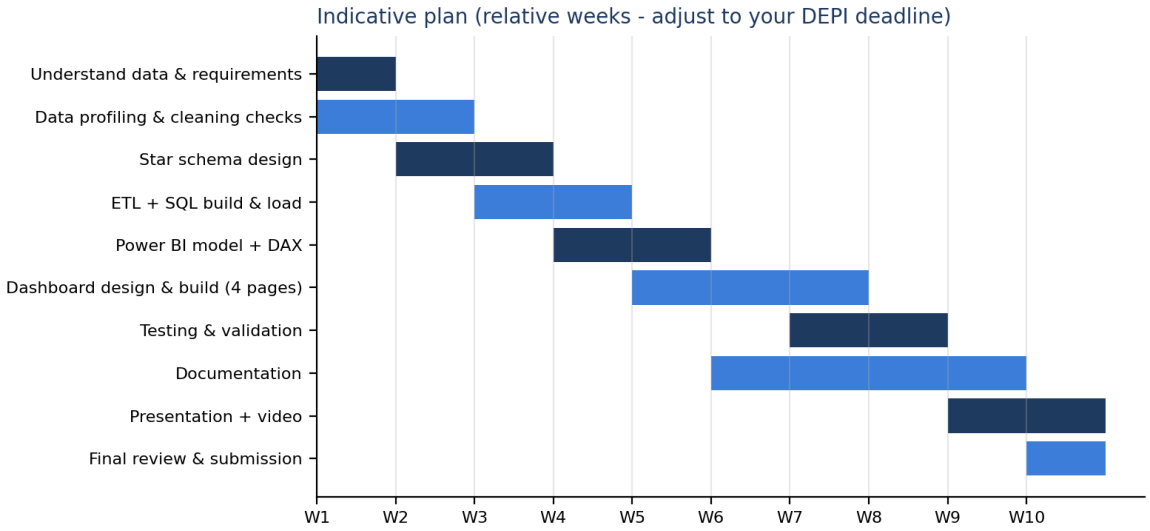
In scope	Out of scope
Data profiling, star schema, ETL (Python), SQL Server DDL/load, DAX measures, 4-page report, documentation, presentation, demo video	Real-time streaming, a production deployment, and a validated failure-prediction model (see section 6 for why)

Key figures (from the data)

Metric	Value
Monitoring records	100,000
Machines	5,000 (20 readings each)
Time window	2026-01-01 to 2026-03-07 (hourly timestamps)
Failure events (machine_failure = 1)	1,472 (1.47% of readings)
Average machine health score	76.4 / 100
Total downtime hours	50,003

2. Project planning

Methodology: a lightweight, phased waterfall with a review at the end of each phase - suitable for a single-person analytics project with a fixed deadline. Weeks below are relative; shift them to match your DEPI dates.



Work breakdown and deliverables

Phase	Main tasks	Deliverable
Understand	Read dataset guide, list business questions, confirm scope	Scope & KPI list
Profile data	Check missing values, duplicates, constants per machine, label balance	Profiling notes (section 6)
Design	Dimensional model, keys, risk rule, naming	ER diagram, data dictionary
Build data layer	Python ETL, SQL Server DDL, bulk load, row-count checks	etl/, sql/ folders
Build report	Model relationships, DAX measures, 4 pages, theme	Dashboard (.pbix)
Validate	Reconcile totals SQL vs Power BI, test slicers, test the risk tier against real failures	Test log
Communicate	Documentation, presentation PDF, demo video	Docs, PDF, video

Tools

Python (pandas) for ETL; SQL Server (T-SQL) for the warehouse; Power BI Desktop + DAX for the report; Git/GitHub for version control; matplotlib/reportlab for the diagrams and documents.

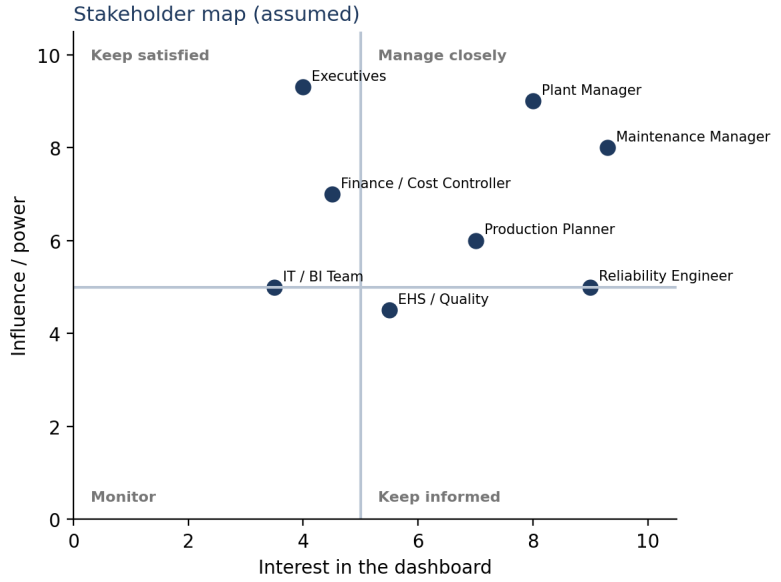
Risks and mitigation

Risk	Impact	Mitigation
Source column maintenance_priority is constant ('Low')	The README's 'critical machines' view is impossible	Derived risk_level with documented thresholds; validated against real failures
Only 1.47% of readings are failures	Any future classifier would be hard to train and easy to over-state	Report rates and counts, avoid accuracy claims; document the imbalance
Sensors do not predict the failure label in this data (section 6)	Cannot promise 'predictive' value	Position the dashboard as monitoring/descriptive; state the limitation openly
Meaning of some fields is undocumented (e.g. maintenance_cost per event or cumulative)	Totals could be misread	Label measures clearly, treat as per-record amounts, flag as an assumption
SQL scripts not run on the author's side	Possible syntax/path errors on first run	Run in a test DB; checks in 03_checks_and_views.sql

3. Stakeholder analysis

Because the dataset is synthetic, the stakeholders below are **assumed roles** that would exist in a real factory group. They drive the choice of KPIs and dashboard pages.

Stakeholder	Main interest	Key questions	Page	Influence	Interest
Plant Manager	Output, downtime, cost	Which factory loses most hours? Is it getting better?	1, 3	High	High
Maintenance Manager	Maintenance workload and effectiveness	Which machines need attention? What does each maintenance type cost?	2, 3	High	Very high
Reliability Engineer	Root causes of failure	Which failure types dominate? Any sensor pattern?	1, 4	Medium	Very high
Production Planner	Capacity and scheduling	Which lines/machine types have most downtime?	1, 3	Medium	High
Finance / Cost Controller	Maintenance spend	Total and per-type maintenance cost; cost per failure	3	Medium	Medium
EHS / Quality	Safe, stable operation	Where are anomaly scores high?	4	Medium	Medium
IT / BI Team	Maintainable data pipeline	Is the model documented and refreshable?	(docs)	Low	Medium
Executives	Portfolio view	One-page status across 10 factories	1	Very high	Medium



Engagement approach: Plant Manager and Maintenance Manager are managed closely (review each page draft); Executives are kept satisfied with page 1; Engineers and Planners are kept informed and invited to give feedback on filters and drill-through.

Functional requirements (derived)

- Filter every page by factory, machine type, date range and shift.
- Show failure count, failure rate, health score, downtime and maintenance cost as KPI cards.
- Show failure events by type and failure rate by factory.
- Provide a drill-through from a risk table to a single machine.
- Refresh from SQL Server without manual editing.

Non-functional requirements

- Page load under a few seconds with 100,000 fact rows (star schema, no unused columns in visuals).
- Colour is never the only carrier of meaning (labels/icons accompany risk colours).
- All measures documented in powerbi/measures.dax.

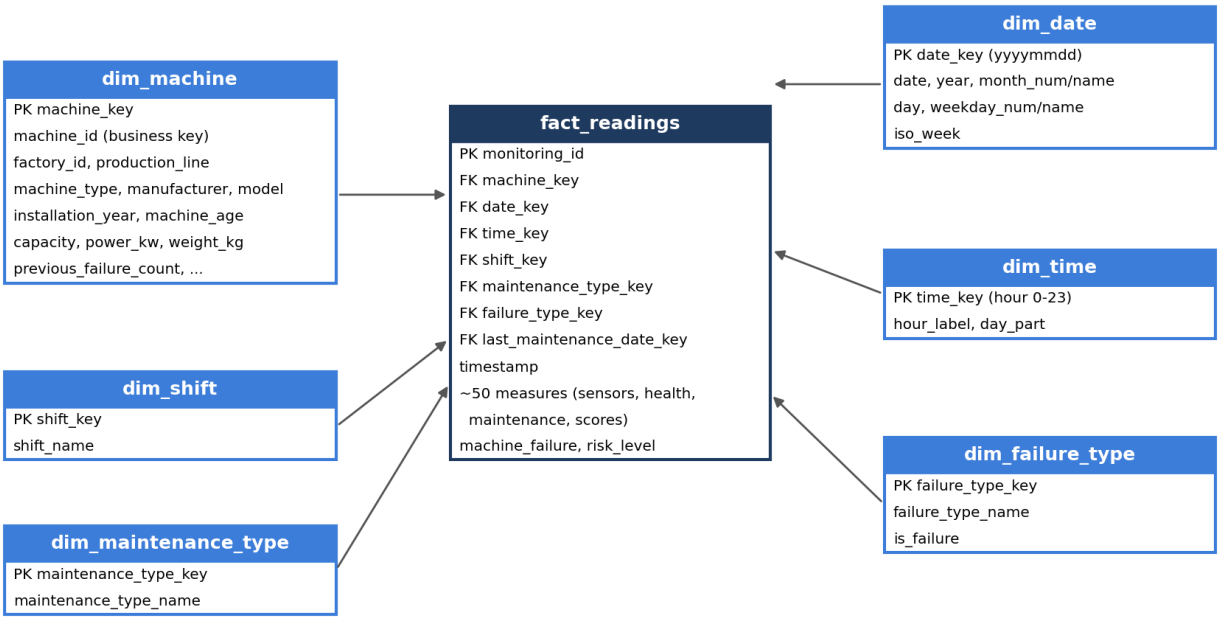
4. Database design

Source profile

One flat file: 100,000 rows x 72 columns, 0 missing values, 0 duplicate rows, `monitoring_id` unique. 14 columns are constant per machine (factory, line, type, manufacturer, model, installation year, age, capacity, power, weight, shift hours, previous failure count, failure frequency, maintenance priority) - these belong in a machine dimension, which the ETL verifies programmatically before building it.

Model: star schema (grain = one monitoring reading of one machine)

Star schema: Smart Manufacturing Data Warehouse



Arrows point from dimension (1) to fact (many). dim_date is also used for last_maintenance_date_key (role-playing).

Table dictionary

Table	Rows	Key	Purpose
fact_readings	100,000	monitoring_id (PK); FKs to all dimensions	All sensor, health, maintenance and target measures + derived risk_level
dim_machine	5,000	machine_key (surrogate); machine_id (business key)	Static machine attributes (factory, line, type, manufacturer, model, size, age)
dim_date	185	date_key (yyyymmdd)	Calendar; also covers last_maintenance_date, which starts in 2025-09 (role-playing date)
dim_time	24	time_key (hour)	Hour of day and day part
dim_shift	3	shift_key	Morning / Evening / Night
dim_maintenance_type	5	maintenance_type_key	Corrective, Emergency, Predictive, Preventive, Routine
dim_failure_type	7	failure_type_key	6 failure types + 'No Failure'; is_failure flag

Design decisions (and why)

- **Surrogate keys** (machine_key) keep joins small and independent of the source ID format.

- **Low-cardinality text columns** (shift, maintenance type, failure type) moved to dimensions so slicers and labels come from one place.
- **Measures stay in the fact**: they vary per reading, so splitting them would add joins without benefit.
- **Derived risk_level** is stored in the fact (computed in the ETL) so SQL and Power BI show the same numbers. Rule: Critical if failure_probability >= 5 or health < 65; High if >= 3 or health < 68; Medium if >= 1.7; else Low. Thresholds are roughly the 75th/90th/99th percentiles of failure_probability - a heuristic, not a validated model.
- **maintenance_priority** (constant 'Low') is dropped from the fact because it carries no information.

Quality checks built into the ETL

- Row count after the split equals the raw row count (100,000).
- Every foreign key exists in its dimension (no orphans).
- Total failures (1,472) and total downtime hours are identical before and after the split.
- Per-machine attributes verified constant before they are placed in dim_machine.

Implementation files: etl/build_star_schema.py (builds the 7 CSVs), sql/01_create_schema.sql (T-SQL DDL with PK/FK), sql/02_load_data.sql (BULK INSERT), sql/03_checks_and_views.sql (checks + 2 views). The SQL scripts were written for SQL Server and have not been executed by the author - run them first in a test database.

5. UI/UX design

Design goals

- Answer the stakeholders' first question within 5 seconds on each page (KPI cards on top).
- Same layout skeleton on all pages: left navigation + slicer panel, KPI row, two or three charts, one detail visual.
- Consistent filters: a sync-slicer group so a factory choice follows the user across pages.

Visual identity

Element	Choice
Primary colour	Navy #1F3A5F (titles, main series)
Accent	Blue #3C7DD9; amber #F2A33A (attention); red #D64545 (critical); green #2E9E83 (healthy)
Typography	Segoe UI; KPI numbers 28 pt semibold, labels 10 pt
Theme file	powerbi/theme.json (import via View > Themes > Browse for themes)

Chart choices

Question	Visual	Reason
How many / how often?	KPI cards	Single number, fast reading
Which failure types dominate?	Donut or sorted bar	6 categories; sorted bar is easier to compare
Which factory is worse?	Bar with a reference line	Compare against the overall 1.47% rate
How is health distributed?	Gauge + histogram	Gauge for the average, histogram for the spread
Where is the cost?	Column by maintenance type	Categorical comparison
Do sensors move together?	Scatter	Relationship between two numeric fields

Interaction

- Slicers: factory, machine type, date range, shift (synced across pages).
- Cross-filtering between visuals on a page; drill-through from the risk table to a machine detail page.
- Tooltips show counts and the formula meaning (e.g. failure rate = failures / readings).

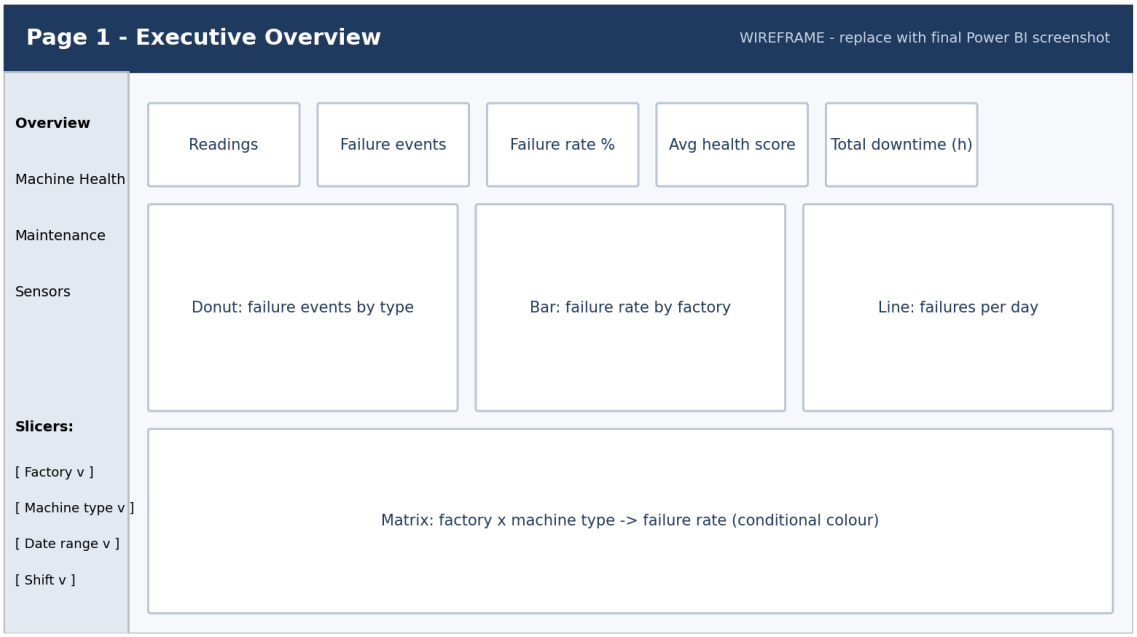
Accessibility

- Contrast of text on white at least 4.5:1 (navy on white).
- Risk levels use labels and ordering as well as colour.
- Alt text on every visual; logical tab order.

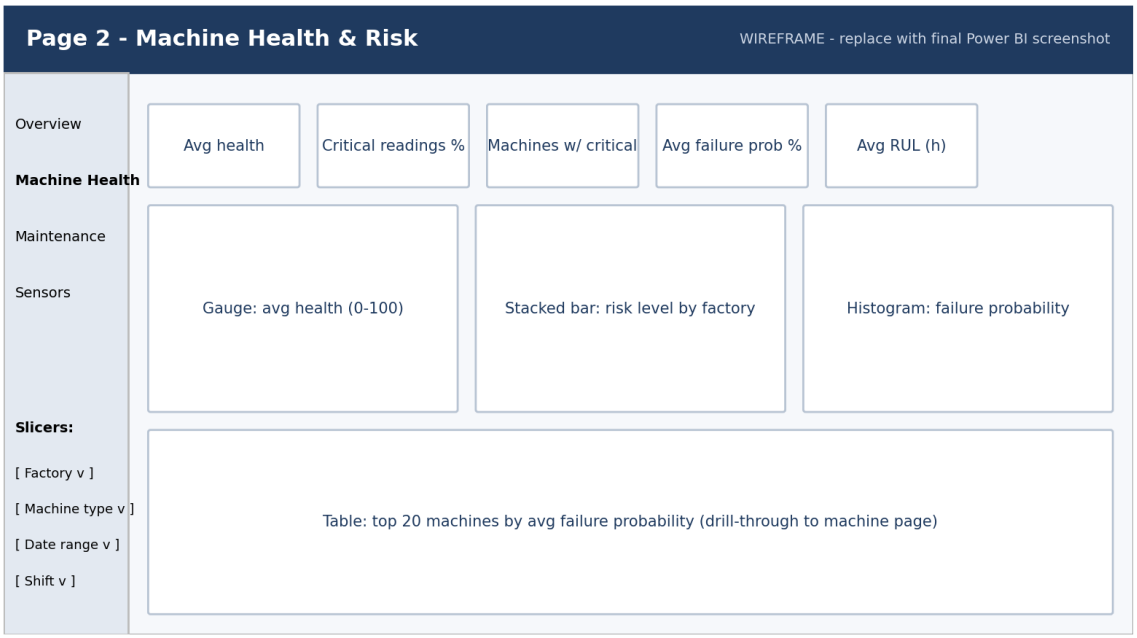
Wireframes

Mock-ups made before building; replace with the final screenshots in the presentation.

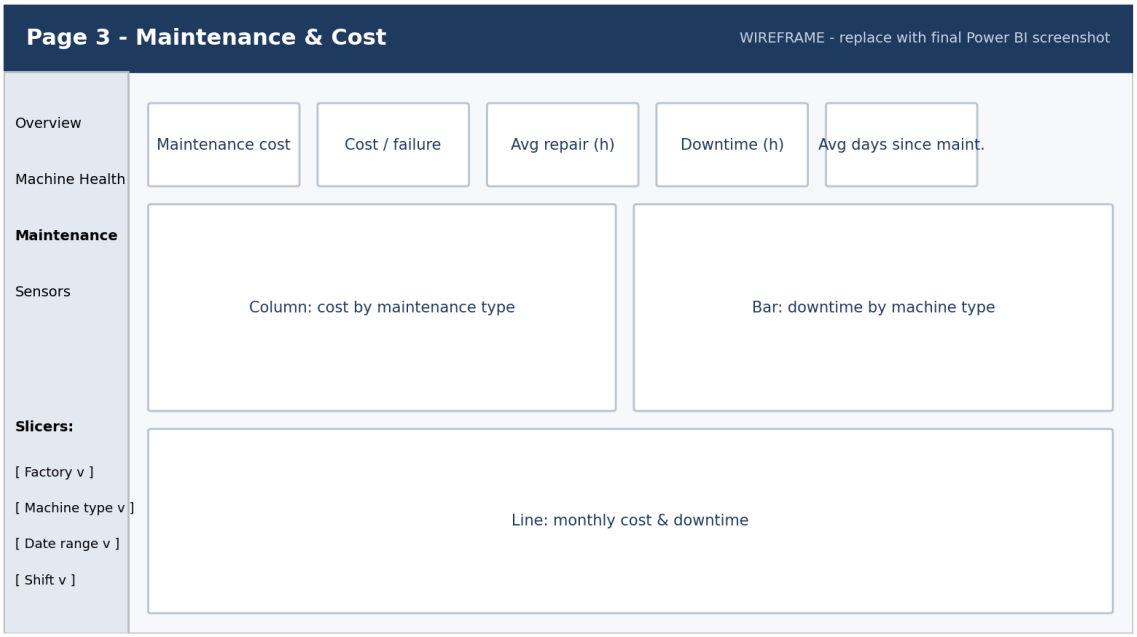
Page 1 - Executive Overview



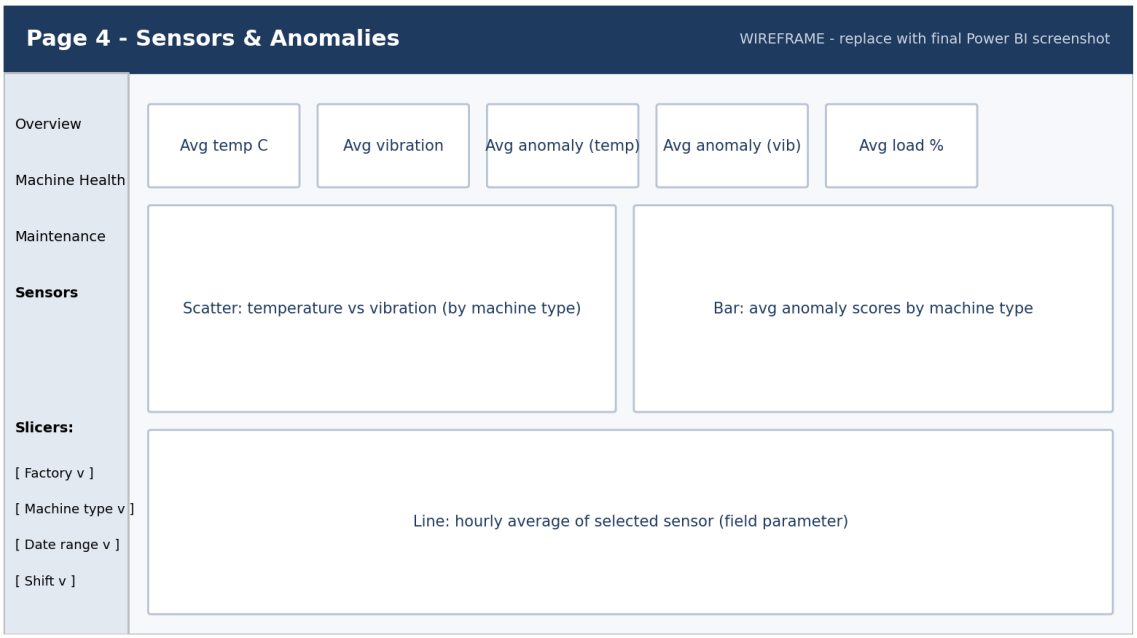
Page 2 - Machine Health & Risk



Page 3 - Maintenance & Cost



Page 4 - Sensors & Anomalies



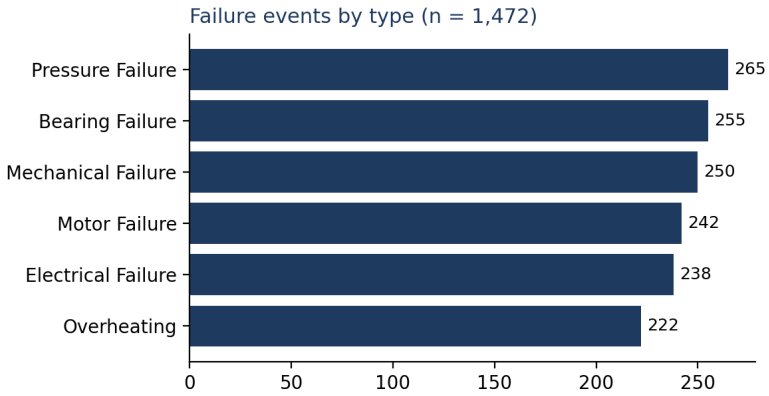
6. Data findings and limitations

These findings come from profiling the file itself. They matter because they change what the dashboard may claim.

6.1 The dataset guide does not fully match the data

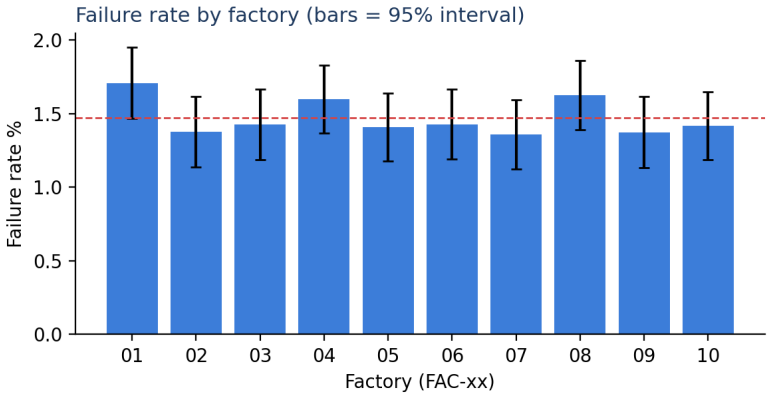
Guide says	Actual data
maintenance_priority table with 'Critical' machines	Constant: 'Low' in all 100,000 rows
8 failure types	6 failure types + 'No Failure' = 7 values
Dataset is ready to use (0 missing, 0 duplicates)	Confirmed: 0 missing, 0 duplicate rows

6.2 Failure events



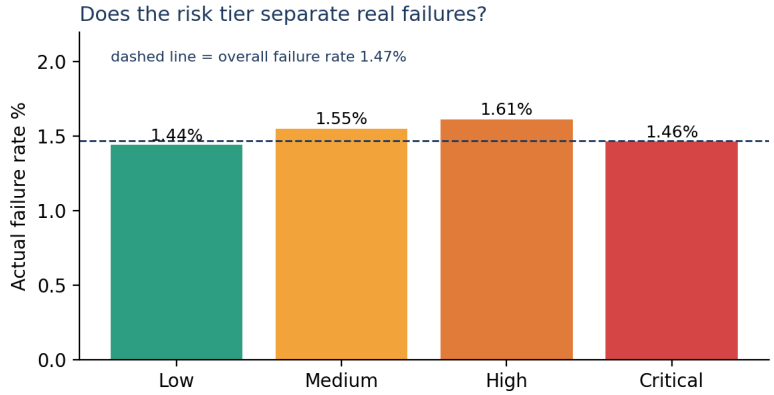
Failures are rare (1.47% of readings) and spread almost evenly across the six types (222 to 265 events each).

6.3 Factory differences are small



Factory failure rates range from 1.36% to 1.71%. A chi-square test of 'failure rate is the same in all 10 factories' gives p = 0.41, so the differences are consistent with chance. The dashboard should not rank factories by failure rate as if the gaps were meaningful.

6.4 The risk tier and probability scores do not separate real failures



Actual failure rate by derived risk level: Low 1.44%, Medium 1.55%, High 1.61%, Critical 1.46% - all close to the overall 1.47%. In addition, no numeric column has a correlation above about 0.007 (absolute) with machine_failure, and a gradient-boosting model cross-validated by machine gave an AUC of 0.505 (0.5 = random guessing).

What this means for the project: the dashboard is a **monitoring and reporting** tool. The failure_probability, machine_health_score and remaining_useful_life fields are shown as provided by the dataset, but this project does not claim they predict real failures. A genuine predictive-maintenance model would need data where the failure label is linked to the sensor history.

6.5 Other caveats

- The dataset is synthetic; conclusions apply to the data, not to a real plant.
- The unit/currency and exact meaning of maintenance_cost, and whether some fields are cumulative, are not documented - totals are treated as per-record amounts (assumption).
- The 20 readings per machine are not evenly spaced in time and cover only about 9 weeks, so long-term trends cannot be judged.

7. How to reproduce

Step	Command / action
1. Get the data	Download the CSV from Kaggle (Smart Manufacturing & Predictive Maintenance) and place it in data/
2. Build the star schema	python etl/build_star_schema.py (creates data/star/*.csv and runs the checks)
3. Create the database	Run sql/01_create_schema.sql in SQL Server
4. Load	Edit the folder path in sql/02_load_data.sql, then run it; confirm with sql/03_checks_and_views.sql
5. Power BI	Get Data > SQL Server (or Text/CSV on data/star), set relationships, add powerbi/measures.dax, apply powerbi/theme.json; follow powerbi/BUILD_GUIDE.md

Repository: link to be added after upload to GitHub.